

PRODUCT OVERVIEW

An operations workspace for transport teams.

Chat-first dispatch, where every conversation is attached to a real transport object — a truck, its trip, its stops, its documents and the people running it.





What follows

01 The case

Why dispatch teams are underserved, and the single idea Dispo-chat is built on.

Pages 3–4

04 Transport operations

Vehicle status, the trip lifecycle, stops and proofs, truck routing, the fleet board.

Pages 10–13

02 The workspace

The company workspace, the cross-company network, and the shape of the app itself.

Pages 5–6

05 Beyond the desk

Files and recall, the company network, and the native Android app for drivers.

Pages 14–16

03 Conversations

Rooms, the message toolkit, and the realtime layer that makes it feel live.

Pages 7–9

06 Under the hood

Architecture, scale, security, the design system, the numbers, and what's next.

Pages 17–23

Dispatch runs on tools that were never built for dispatch.

A transport office coordinates trucks, drivers, customers and partners in real time — and does it across four apps that know nothing about each other.



The thread is everywhere

WhatsApp for the driver, email for the customer, a phone call to the yard, a spreadsheet for the plan. No load ever has one home.



Context dies with the message

A plate, a CMR photo, a delivery window, a price — buried in an untyped thread with no structure, no status and nothing to search.



The truck is a black box

Status lives in someone's head. The answer to "where is it?" is another phone call, and it goes stale the moment it's given.

Each of these is a **conversation** problem before it is a software problem — which is why bolting a chat tab onto a TMS has never fixed it.

DISPO-CHAT IS BUILT ON ONE IDEA

Every conversation is attached to a real transport object.

Open a room in Dispo-chat and you are not opening a thread — you are opening a truck. Its plates, its driver, its current status, its trip, every stop on that trip, the documents proving each one, and the people responsible. The chat is the interface; the operation is the record. Nothing has to be re-entered somewhere else to count.

CHAT-FIRST

People already coordinate by message. The product meets that habit instead of fighting it — then makes the messages structured.

OPERATIONALLY TYPED

A room has a type. A vehicle room carries status, a trip, stops and proofs that the interface understands and can act on.

A NETWORK, NOT A SILO

Many carriers share the platform. Inside your company chat is open; across companies it is gated by an explicit handshake.

One workspace per company. One network between them.

Signing up creates a company workspace and its first admin. Everything — people, vehicles, places, conversations — is scoped to that workspace and gated by membership on every single query.



Inside the workspace

Open by default

- Colleagues are **trusted contacts** — no connection request needed to DM anyone in your own company.
- A live **Company** roster in the sidebar, kept in sync over a workspace-wide socket room.
- Members join by **email invite** with a pre-assigned role, or by an invite link.
- Shared **saved places** — depots, customers, borders — reusable across every trip.



EXPLICIT
HANDSHAKE



Across companies

Gated by consent

- A **company directory** to find carriers and partners already on the platform.
- Request → accept → decline, LinkedIn-style, with realtime notification on every step.
- A cross-workspace DM is **impossible** until a connection is accepted — enforced server-side, not in the UI.
- Public **company profiles** so a partner knows who they're accepting.

ROLES

Admin

Dispatcher

Driver

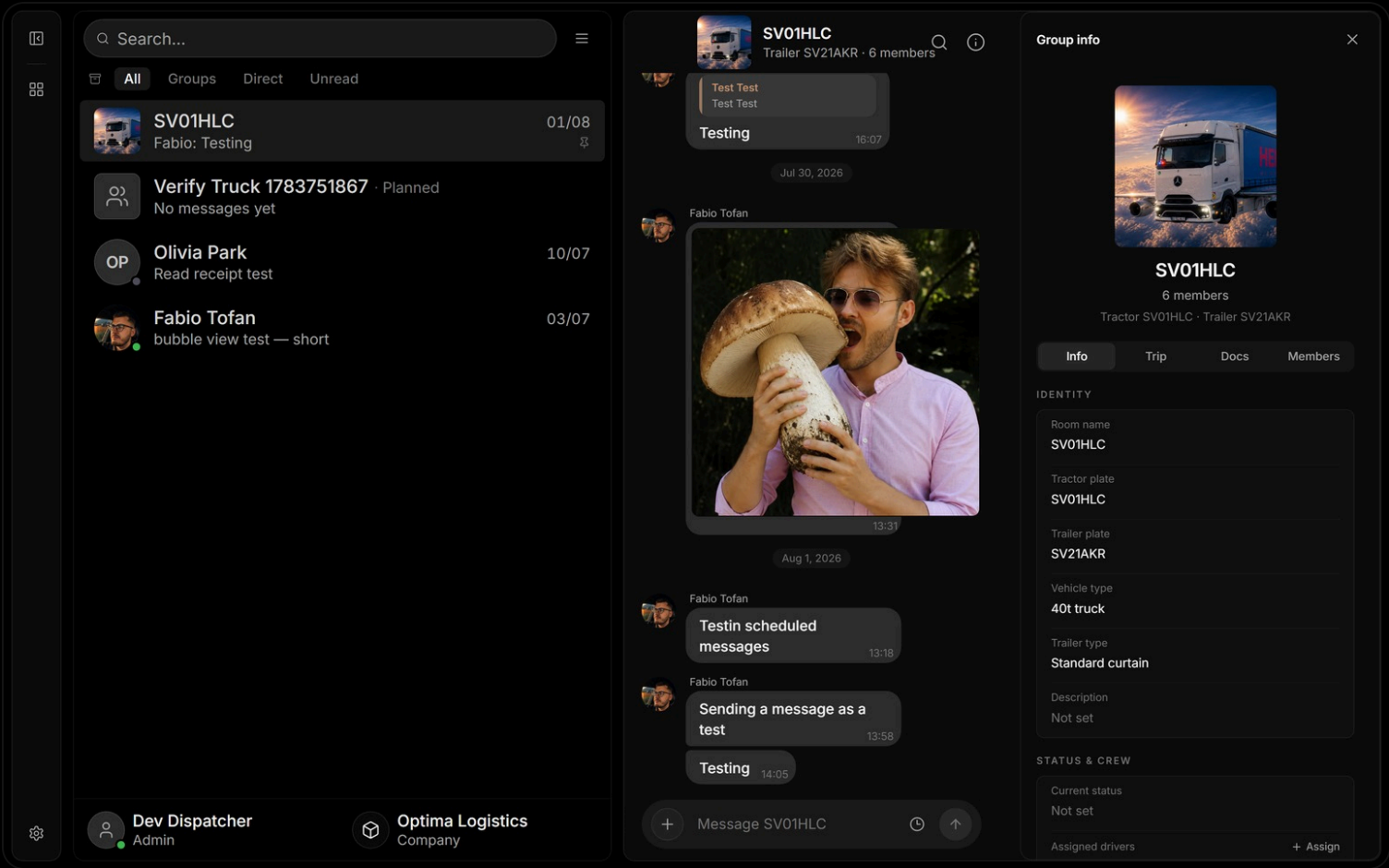
Accountant

Member

Role is assigned at invite time and decides what a member can see and change.

Three columns, one job each.

Navigate, converse, inspect. The rail drills in rather than floating menus over the app, so you never lose your place.



• LIVE CAPTURE

- A **Rail — pure black**
A drill-in stack: list → account → profile. Conversations grouped by type, with live status inline.
- B **Conversation**
Header, message column and composer. The only surface that ever holds the operational thread.
- C **Inspector**
Group info, tabbed into Info, Trip, Docs and Members — opening beside the chat on the same tone, separated by edge, not colour.

Screenshot of the running build. Content is this workspace's own development data.

A room is a thing, not a thread.

Two room types are live today, and they behave differently because they mean different things.



Vehicle rooms

A truck and everything happening to it

The centre of the product. Created from the workspace with tractor and trailer plates, then it accumulates the operation.

- **Vehicle** — plates, capacity, notes, current status.
- **Trip** — order reference, client, route, and a status walked through 15 stages.
- **Stops** — typed waypoints with windows, addresses and completion state.
- **Members** — dispatchers plus the assigned driver, each with a room role.
- **Documents** — the transport paperwork the load is expected to produce.



Direct messages

People, inside and outside the company

One-to-one, with the trust boundary built into the room itself rather than bolted on as a setting.

- **Same workspace** — open immediately, no request, no ceremony.
- **Different workspace** — requires an accepted connection; the server refuses to create the room otherwise.
- **Profile preview** — tap any avatar for the person's role, company and contact detail.
- **Photo identity** — DMs show the person; vehicle rooms show a truck glyph, so the list is scannable at a glance.

PER-USER, PER-CONVERSATION

• Pin

• Mute

• Archive

• Mark unread

• Delete for me

Your rail is yours. Archiving a room for yourself doesn't touch anyone else's view of it.

Everything you expect from a modern messenger — and it all persists.

Not a demo chat. Edits, deletes, pins and forwards are real endpoints with real broadcasts, membership-gated on the server.

- | | | | |
|---|---|--|---|
|  Reply & quote
Threaded context inline, with a tap back to the original. |  Edit & delete
Edit history flagged; deletes tombstoned, never silently vanished. |  Pinned messages
A pinned bar above the thread for the reference everyone keeps asking for. |  Forward
Push a message into any other room you're a member of. |
|  Mentions
@people and @trips, stored as first-class rows with their own badge count. |  Scheduled send
Write it now, deliver at a chosen time — a server worker does the sending. |  Attachments
Drag, paste or pick. Images and PDFs preview before they send. |  Search
In-conversation find with highlight-and-jump, over a dedicated index. |
|  Read receipts
Per-member, with a panel showing exactly who has seen it and when. |  System messages
Status changes, joins and assignments narrated into the thread itself. |  Drafts
Per-room, kept locally, restored when you come back to the room. |  Two display modes
Bubbles or a plain stream; compact rows or avatar rows. Your call. |

History is **cursor-paginated** off a dedicated (`group_id`, `created_at desc`) index, so scrolling back through a year of a truck's life stays fast — and messages de-duplicate by id, so a reconnect never doubles the thread.

Nothing needs a refresh.

Dispatch is a real-time job. Twenty-seven distinct events travel over a single authenticated socket, so what one person changes, everyone sees happen.

Presence

Who is online right now, snapshotted on connect and patched on change — with an idle detector so "online" means it.

Typing

Relayed per room, throttled, and shown in the header as well as the thread.

Unread & mentions

Denormalised counters, so the rail badge is a read, not an aggregate scan.

Notifications

Web Push to the device, browser notifications on desktop, plus an optional sound.

Reconnection is handled by the transport with backoff, and every payload carries an id — so a tunnel, a lift or a dead zone costs you latency, never a message.

REALTIME EVENT SURFACE

27 events

MESSAGES

message:newmessage:editedmessage:deletedmessage:hiddenmessage:pinnedmessage:unpinnedmessage:system

ROOMS

group:addedgroup:removedgroup:readgroup:unreadgroup:prefs

PEOPLE

presence:snapshotpresence:updatetypingconnection:requestedconnection:acceptedconnection:declined

OPERATIONS

trip:statustrip:updateddriver:locationattachment:preview

Every socket is authenticated with the same httpOnly cookie as the REST API and joins only the rooms its user is actually a member of. There is no client-declared subscription.

The truck has a status. So does everyone's understanding of it.

Vehicle state is set by hand, on purpose. It is what a dispatcher knows — not what a tracker guesses — and it shows up everywhere the vehicle appears.

VEHICLE STATUS

Available Driving Loading Unloading Waiting Break Service Completed

One tone, everywhere

A status colour is defined once and reused by the sidebar row, the chat header, the trip strip and the fleet board. Nothing drifts.

Vehicle record

Tractor and trailer plates, capacity and free-form notes live on the room and drive the truck-legal routing profile.

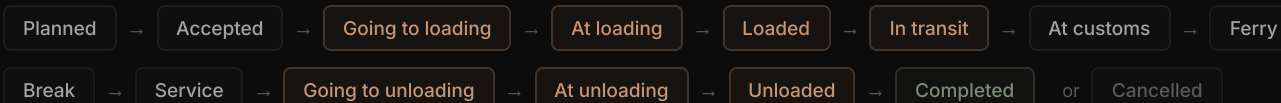
Assigned drivers

The driver assignment is what unlocks the mobile trip view and gates every driver endpoint on the server.

 **A deliberate constraint.** Nothing about vehicle or trip status is inferred from GPS. Location is shown when a driver's phone reports it, but it never silently rewrites a status a human set — the operational record stays something a person is accountable for.

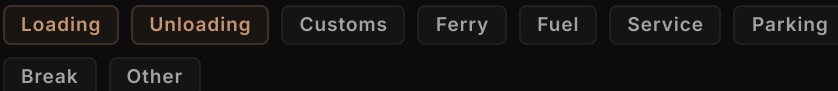
Fifteen stages, in the order a load actually moves.

Not "open / closed". The lifecycle names the states a European road load really passes through, so a glance at the rail answers the question without a phone call.



STOPS

A trip is a list of typed waypoints, each with an address, a time window, and its own completion state — planned, done or cancelled.

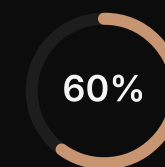


Addresses are picked from HERE search or from the workspace's saved places, so the same depot is the same coordinates every time.

PROOF OF TRANSPORT

Before a driver leaves a loading stop — or completes an unloading one — the app asks for the paperwork.

- JPEG, PNG or PDF, up to 25 MB, checked server-side.
- Written to an **append-only** table, tied to the trip, the stop, the kind and the uploader.
- Never edited, never overwritten — an audit trail, not a file field.



Progress at a glance

Completed stops over total, drawn in the status tone, on the strip above every vehicle conversation.

Routes a truck can legally drive.

Consumer map APIs will happily send a 40-tonne artic under a 3.8 m bridge. Dispo-chat routes on HERE's truck profile, with the vehicle's real dimensions attached to the request.

Truck routing

Height, width, length and weight taken from the vehicle record and sent as routing constraints.

Toll estimation

Per-charge breakdown across every operator on the route, with transponder warnings where one is required.

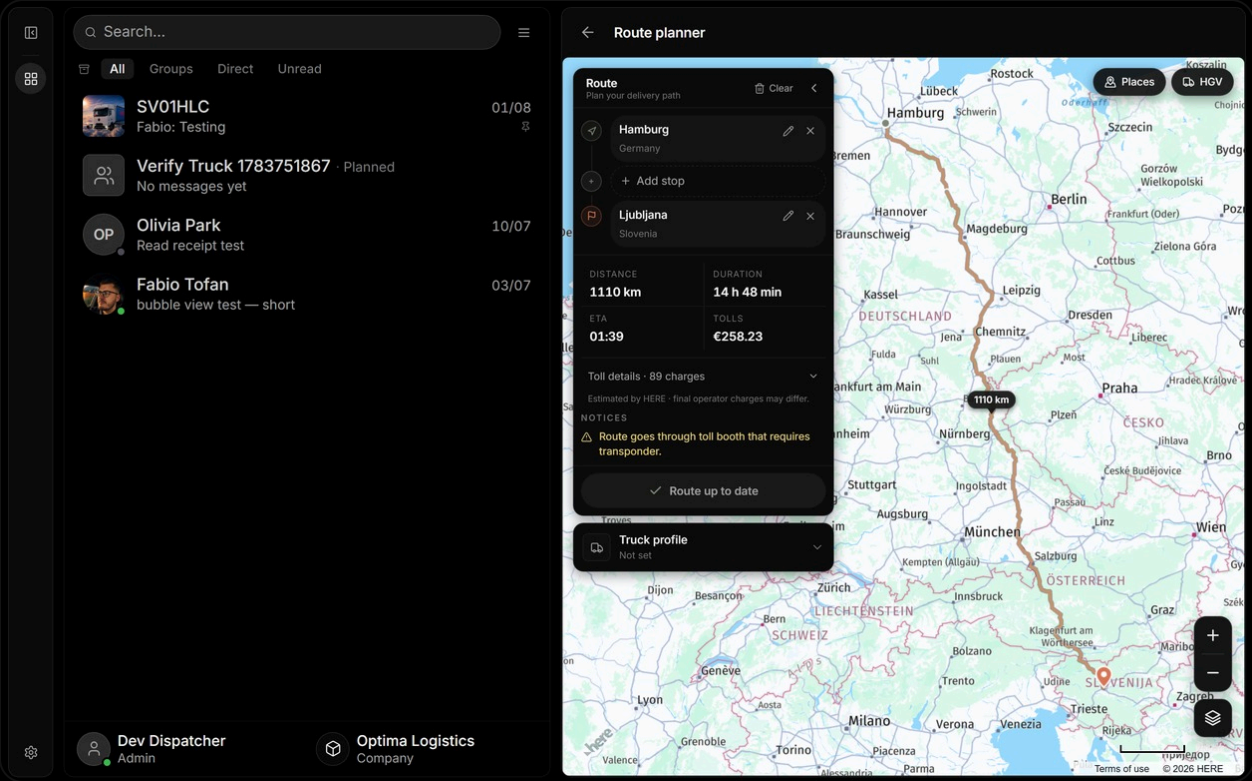
Place search

Autocomplete, reverse geocode and road-snapping for stop addresses.

Saved places

Depots, customers and crossings saved once, shared by the whole workspace.

🔒 Every HERE call is proxied through the API under /api/here/*. The key never reaches a browser or a phone.

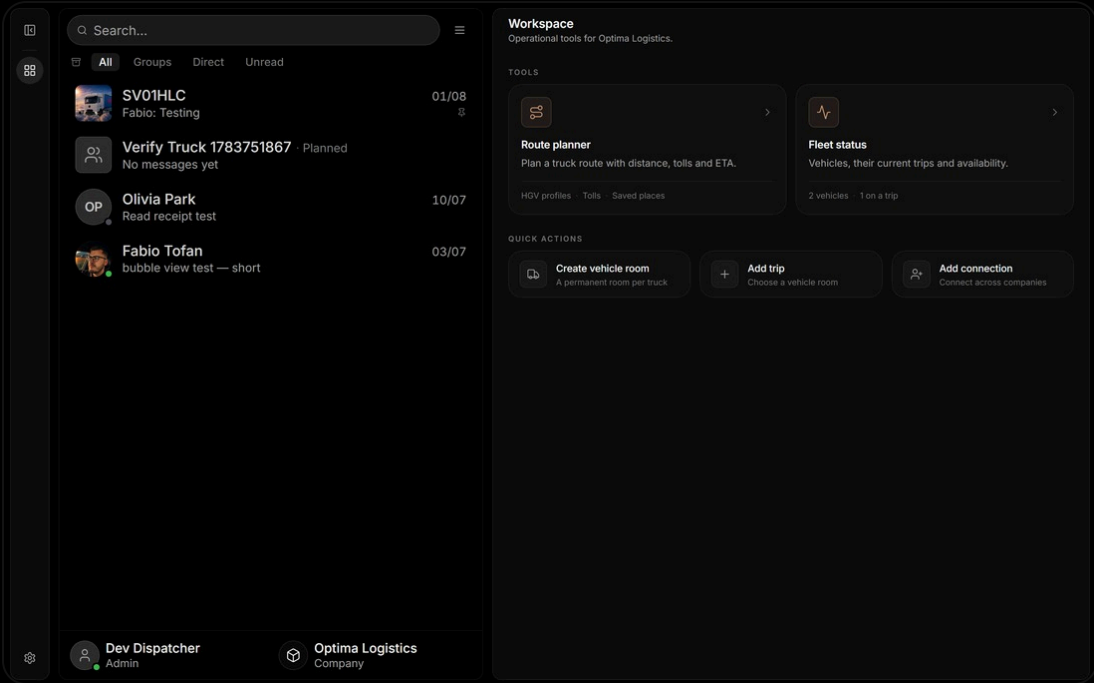


- LIVE CAPTURE Hamburg → Ljubljana on the HGV profile: 1 110 km, 14 h 48 min, €258.23 in tolls across 89 separate charges.

Chat is where the work happens. Tools are where you check it.

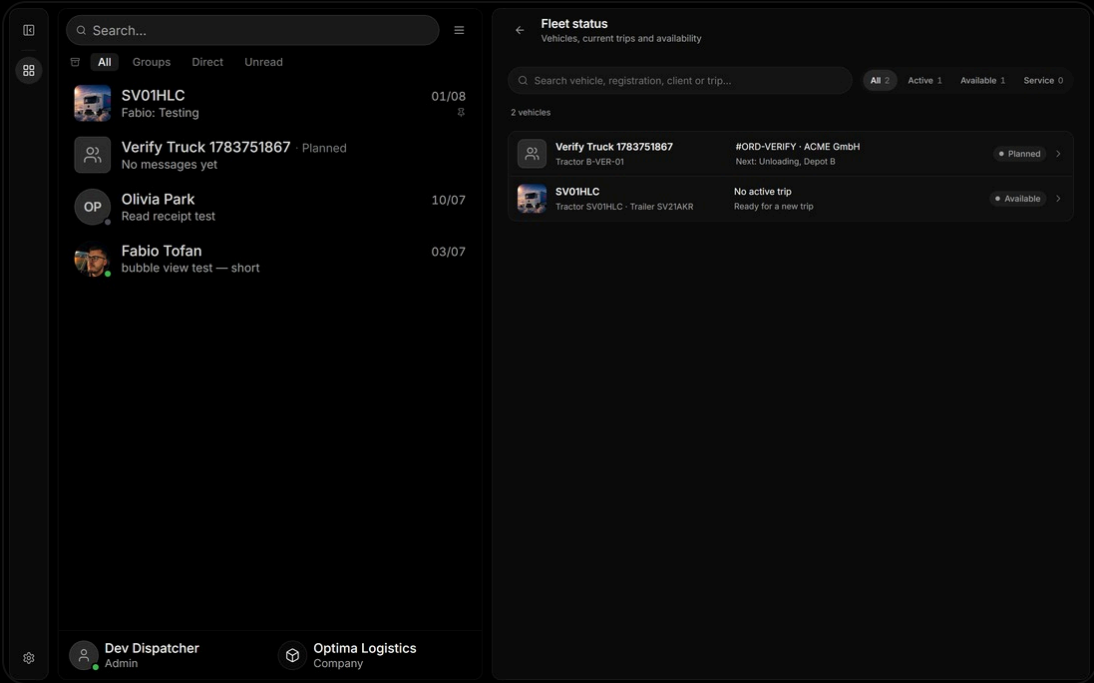
A hub beside the conversations: plan a route, or see every truck, its current trip and its availability on one board.

● LIVE CAPTURE



The tools hub

Route planner and Fleet status beside quick actions for the three things a dispatcher starts most often.



The fleet board

Every vehicle with its trip, next stop and status — filterable, and searchable by plate, client or order.

Screenshots of the running build against this workspace's own development data — a two-vehicle test bench, not a production carrier. Every row is assembled from the same trip summary the sidebar and chat header read, so the board can never disagree with the room.

Paper is the product.

Transport runs on documents — CMR, POD, transport orders, delivery notes, invoices. Attachments are treated as infrastructure, not a nice-to-have.



Send anything

Drag onto the room, paste, or pick from the attach menu — with a caption bar to say what it is before it goes.



Previews that render

Images get a generated preview; PDFs render to a real first-page thumbnail and open inline, without a download.



Off the request path

The upload stores, posts and returns. Image work runs as a queued job and the preview arrives over the socket when it's ready.



Private by default

Bytes stream through the API, gated on room membership. A file URL is not a capability — there are no public object links.

FINDING IT AGAIN

Message search

A dedicated search index over message bodies, with in-conversation highlight and jump.

Attachment tab

Every file a room has ever carried, in one grid, separate from the chat scroll.

Pinned

The reference number or the loading instruction, held above the thread.

Directory search

Companies and people, across the whole network, from the same field.

DOCUMENT TYPES

In progress

Transport Order

CMR

POD

Delivery Note

Invoice

Other

Room-level document filing is laid out and typed in the UI; it will reuse the same upload and preview pipeline rather than growing a second one. Trip proofs at loading and unloading already store this way today.

Who you are is part of the product.

In a network where strangers do business, a profile is not decoration — it is the thing a partner decides on. Five profile surfaces share one visual language and one permission rule.



My profile

Photo, name, role, contact detail and availability.



User preview

The same surface, read-only, for anyone else you meet.



Company profile

Legal name, fleet size, lanes and dispatch contact.



Account

Sign-in, verification and the reference for all five.



Workspace settings

Members, roles, invites, places and appearance.



Identity you can rely on

- Display name, e-mail and company name are **immutable after signup** — you cannot become someone else after a partner has connected with you.
- Legal name and dispatch e-mail **lock once set**, so they can be filled in later but not quietly swapped.
- Enforced **server-side** on every write, not just greyed out in the interface.
- E-mail addresses are **verified** by token before an account is trusted.



Getting people in

- **Workspace invites** — e-mail a colleague with the role already chosen; they land straight into a registration form bound to that invite.
- **Group invites** — bring a partner into a single vehicle room without giving them the rest of your workspace.
- **Connection requests** — the cross-company handshake, with an inbox for pending requests.

The driver isn't at a desk.

A native Kotlin and Jetpack Compose app — not a wrapped web view — sharing the same API, the same socket and the same design tokens as the desktop workspace.

Full conversations

Desktop-parity bubbles, typing, read receipts, mentions, attachments and live messages over the same socket.

Trip view

The assigned driver sees their active trip, its stops and its status — and can move it forward from the cab.

Location reporting

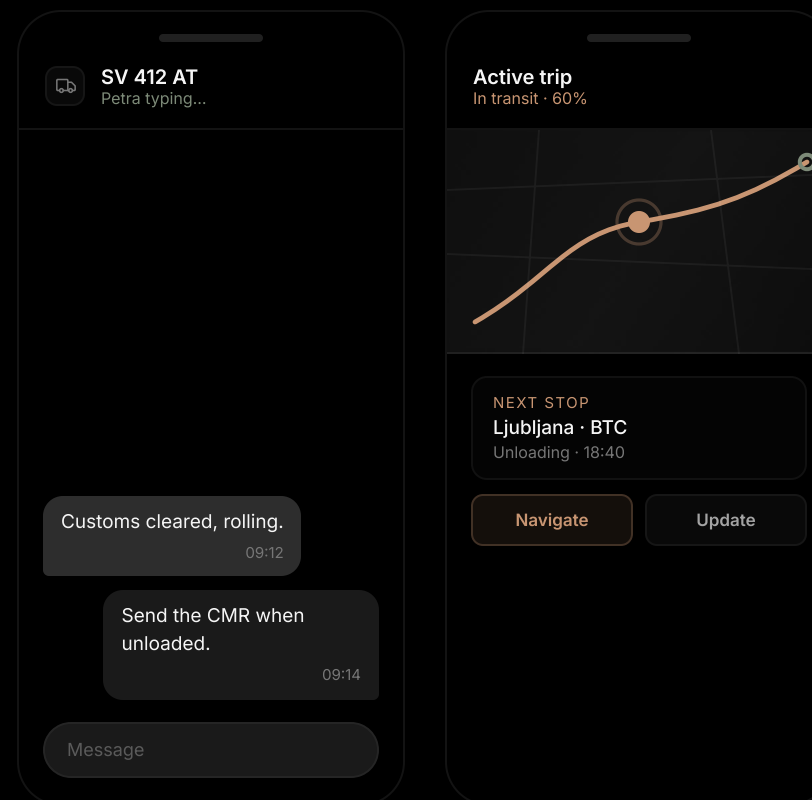
A foreground ping publishes position to the room, so the dispatcher's map marker is current without a phone call.

On-device navigation

A native HERE map with a truck route and a live GPS puck, with hand-off to the phone's navigation app as a fallback.



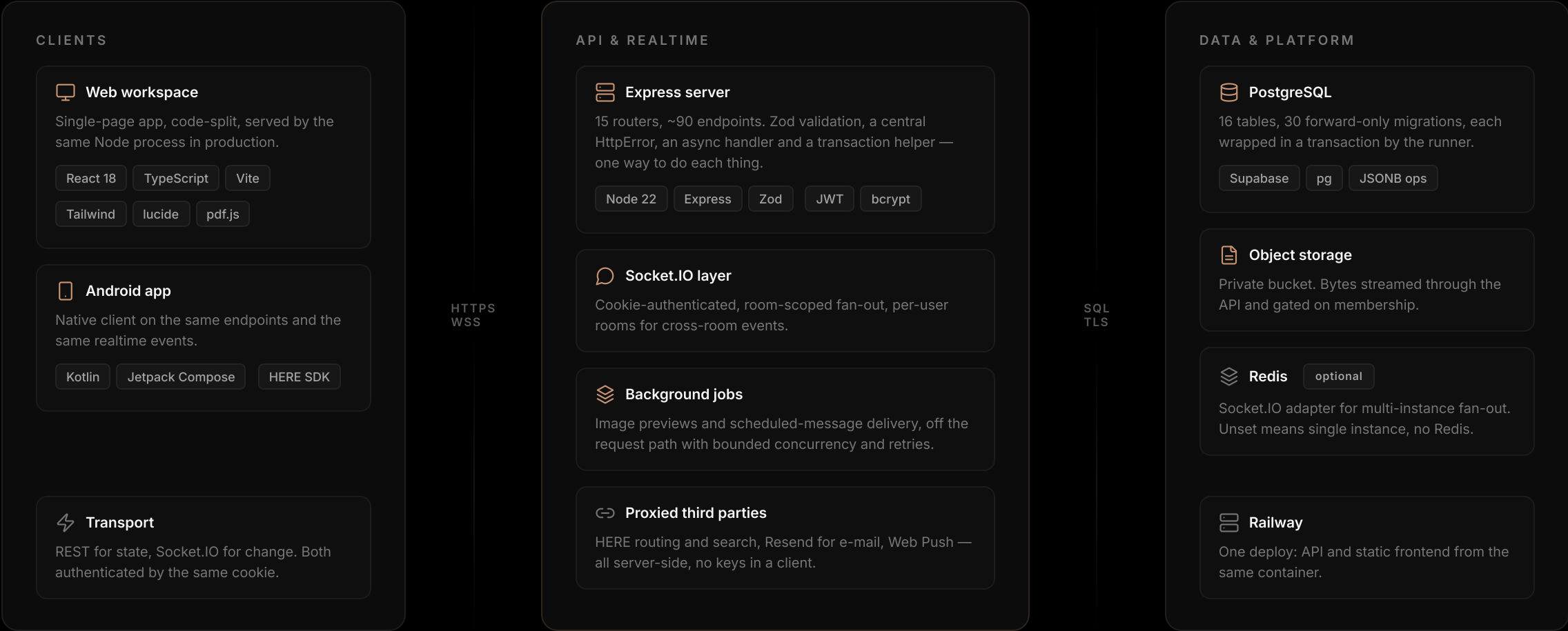
Every Android surface is drawn on **pure black**. On the OLED panels drivers actually carry, an interface that is mostly background costs almost nothing to display — which matters on a twelve-hour shift.



ILLUSTRATIVE MOCKUP — NOT A DEVICE CAPTURE

Boring where it should be.

One API, one database, one socket. No service mesh, no message bus, nothing that needs a diagram to explain a bug at two in the morning.



Built to add a second instance, not to need one on day one.

The realtime layer is the part that breaks quietly when you scale a chat app. It was designed for that first, and the switch is a single environment variable.



Multi-instance fan-out

Sockets pin to one instance. Without a shared adapter, an event emitted on A never reaches a member on B — and it fails silently.

- Redis adapter wired through **every** realtime path, so there are no call-site exceptions.
- Membership changes fan out too — a user's sockets join and leave rooms wherever they live.
- Unset in dev: in-memory. Set in prod: shared — and it **refuses to boot** if it can't connect.



Query & index discipline

The message table grows without limit, so paging it is a first-order concern.

- A full (**group_id, created_at desc**) index keeps cursor pagination index-backed, tombstones and all.
- Redundant indexes dropped to cut write amplification on the hot path.
- Unread counts **denormalised** — the rail badge is a read, not an aggregate.
- A documented **concurrent** index build path for already-large tables.



Observability

Structured JSON lines on the paths that actually tell you something when it's slow — and never a message body, cookie, token or file byte.

- **http_request** — route, status, duration, user and group.
- **preview_job** — status and duration for every image job.
- **socket_connect / disconnect** — with a live connection count.
- **redis_adapter_*** — the scaling posture is visible at boot, not inferred.

KNOWN AND TRACKED

The preview queue is in-process today — jobs queued at shutdown are lost, which is exactly why a re-runnable backfill command exists. A durable driver sits behind a config flag and would change one file.

A multi-tenant chat app is a permission system with a text box.

Companies that compete with each other share this database. Isolation is not a feature of the interface; it is a property of every query.



Authentication

Stateless JWT in an **httpOnly**, **sameSite** cookie, secure in production. Passwords hashed with bcrypt. The unused session table was dropped rather than left lying around.



Input & abuse

Zod validation on every payload. Rate limits on sign-in, message creation, room creation and uploads. Upload size and MIME type checked on the server, not the client.



Attachment access

No public object URLs. Bytes are streamed through a membership-gated route with immutable cache headers, so a leaked link is not a leaked file.



Authorization

Every read and write is scoped by workspace or room membership in the query itself. Driver endpoints are gated twice — member *and* assigned — and answer 403 or 404, never data.



Deletion without destruction

Users are **anonymised, never physically deleted** — a departed colleague's messages stay readable in context for the people still working the load, without their personal data surviving.



Honest failure

Unmatched API paths answer as the API, never as the app — so a renamed endpoint returns a 404 instead of an HTML page a client cheerfully decodes as corrupt data.

Still open, and named as such: a CSRF review for state-changing requests, a global one-account-per-e-mail identity model before public launch, anti-abuse limits on connection requests, and a scripted test harness for the auth path.

Depth by tone, not by shadow.

This deck is drawn from the same tokens as the app. The palette, the corner radii and the type are literally the product's.

THE SURFACE LADDER

- #000000

Rail
The deepest surface — navigation
- #0C0C0C

Chat & panels
The conversation and everything beside it
- #161616

Floating chrome
Menus and popovers only — they must lift
- #1A1A1A

Composer & own bubble
One family, one tone
- #2D2D2D

Incoming & hover
The only step that reads as "someone else"

Panels sit on the chat's exact tone and are told apart by edge and elevation — never by re-tinting a surface. Hairlines at 8% white do the rest; there is not a single drop shadow in the workspace chrome.

ONE RADIUS SCALE, ONE TYPE SCALE

- chip

4
- btn

6
- card

8
- modal

10
- panel

12

Radius is a role, not a taste. There are no arbitrary corner values anywhere in the codebase — a component picks the token that matches what it *is*.

Inter

400 · 500 · 600 · 700

Three text tones and nothing else: **primary** · muted · faint. Colour carries meaning (**active**, done, **alert**) and is never decorative.

SCALES WITH THE SCREEN, NOT WITH A ZOOM CONTROL

One root variable drives every rem-sized surface — no CSS transform, no viewport units, so layout pixels stay real and canvas maps stay exact on a 4K panel.

Compact

Laptops · 0.875×

Default

1080p · 0.9375×

Comfortable

QHD · 1.125×

Ultra-wide

4K · 1.1875×

Where it stands.

Measured against the repository on 2 August 2026.

42_k

LINES OF TYPESCRIPT

Across 229 source files, web and server, fully typed end to end.

~90

REST ENDPOINTS

Behind 15 routers, each validated and membership-scoped.

27

REALTIME EVENTS

One authenticated socket carrying messages, presence and operations.

30

MIGRATIONS

Forward-only, transactional, over 16 live tables.

15

TRIP STAGES

Plus 8 vehicle states and 9 stop types — a real dispatch vocabulary.

46

KOTLIN FILES

A native Android client, not a wrapper, sharing the API and the tokens.

129

COMMITTS

19 May – 2 August 2026. Seventy-six days from empty repo to this.

2

PLATFORMS, ONE SYSTEM

Web and Android drawn from the same palette, radii and type scale.

Counts are of shipped, running code — routers mounted, migrations applied, events emitted. Nothing here is planned work.

The next loop is the network.

Internal dispatch works. The unfinished half is the reason a company would rather be here than in a group chat — doing business with the company next door.

Near Close the network loop

Discover a company, connect, and trade.

- A searchable **company directory** with public profiles as the front door.
- Room-level **document filing** — CMR, POD, invoices — on the existing upload pipeline.
- The global **one-account-per-e-mail** identity model, before public launch.
- **Anti-abuse** on connection requests: limits, blocking, reporting.

Then The commercial object

Turn the conversation into a transaction.

- **Shipment and quote rooms** alongside vehicle rooms — price, currency, expiry, status.
- Milestones from **quoted** to **invoiced**, so a load has a commercial life, not just a physical one.
- **Full-text search** across rooms, references, plates, companies and places.
- Filters by status, date, customer, partner and vehicle.

Ongoing Production hardening

Named, tracked, unglamorous.

- **Durable job queue** behind the flag that's already reserved for it.
- **Log shipping and error monitoring**, then latency and failure-rate alerts.
- **CDN-fronted attachments** to move egress off the app instances.
- **CI** for build, lint, migrations and tests; a scripted auth harness.
- **Backups, retention and data export** decided before external users, not after.



The roadmap is kept in the repository and reconciled against the code, so an unfinished item is visible in the plan rather than implied by its absence from the app.

The conversation was always where the work happened. Now it's where the record lives.

Dispo-chat is a running product: a web workspace, a native Android client, a hardened API and a real operational model — built in seventy-six days, and honest about the parts that are still being built.

